

Entity States in Entity Framework

In this article, I will discuss the **Entity States in the Entity Framework with Examples**. Please read our previous article where we discussed [Entities Types in Entity Framework](#).

Entity States in Entity Framework

The Entity Lifecycle in Entity Framework describes the process in which an Entity is created, added, modified, deleted, etc. Entities have many states during their lifetime. Entity Framework maintains the state of each entity during its lifetime. Each entity has a state based on the operation performed on it via the context class (the class which is derived from DbContext class). The entity state is represented by an enum called EntityState in EF 6 with the following signature.

```
namespace System.Data.Entity
{
    ...public enum EntityState
    {
        ...Detached = 1,
        ...Unchanged = 2,
        ...Added = 4,
        ...Deleted = 8,
        ...Modified = 16
    }
}
```

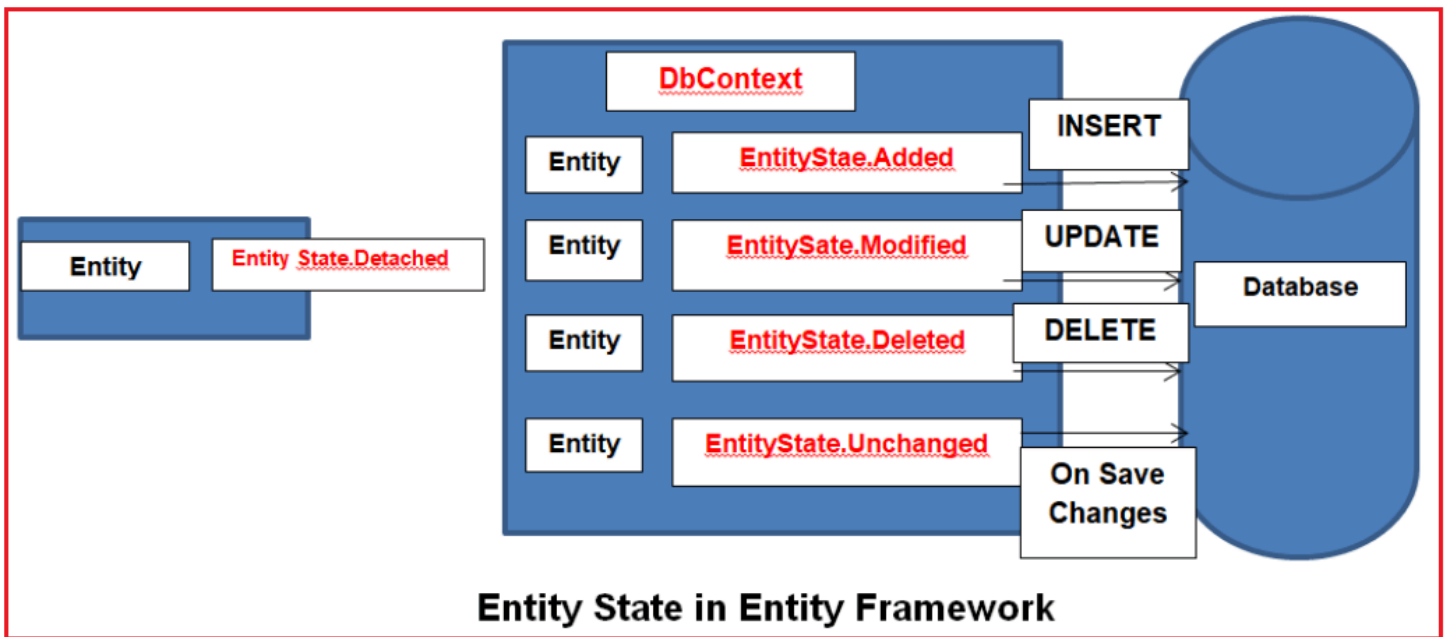
That means the Entity State represents the state of an entity. An entity is always in any one of the following states.

1. **Added:** The entity is marked as added. The entity is being tracked by the context but does not yet exist in the database.
2. **Deleted:** The entity is marked as deleted. The entity is being tracked by the context and exists in the database, but has been marked for deletion from the database the next time SaveChanges is called.
3. **Modified:** The entity has been modified. The entity is being tracked by the context and exists in the database, and some or all of its property values have been modified.
4. **Unchanged:** The entity hasn't been modified. The entity is being tracked by the context and exists in the database, and its property values have not changed from the values in the database.
5. **Detached:** The entity is not being tracked by the context. An entity is in this state immediately after it has been created with the new operator or with one of the System.Data.Entity.DbSet Create methods.

The Context object not only holds the reference to all the entity objects as soon as retrieved from the database but also keeps track of entity states and maintains modifications made to the properties of the entity. This feature is known as Change Tracking.

Entity Lifecycle in Entity Framework

The change in the entity state from the Unchanged to the Modified state is the only state that's automatically handled by the context class. All other changes must be made explicitly by using the proper methods of DbContext class. The following diagram shows the different states of an entity in the Entity Framework.



Let's discuss different states.

Unchanged State of an Entity in Entity Framework with Example

The property values of the entity have not been modified since it was retrieved from the database. SaveChanges ignores this entity. This is the default state the entities will be in when we perform the query and also whenever we attach an entity to the context using Attach() method.

```
using System;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Fetch the First Department
                Department firstDepartment = context.Departments.Find(1);

                //Check the Entity State Before Modifying
                Console.WriteLine($"Before SaveChanges Entity State:
{context.Entry(firstDepartment).State}");
                context.SaveChanges();
                //Check the Entity State After Calling the SaveChanges Method
                of Context Object
                Console.WriteLine($"After SaveChanges Entity State:
{context.Entry(firstDepartment).State}");
                Console.ReadKey();
            }
        }
    }
}
```

Output:

```
Before SaveChanges Entity State: Unchanged
After SaveChanges Entity State: Unchanged
```

Attaching an Existing Entity to the context

If you have an entity that you already know exists in the database but which is not currently being tracked by the context then you can tell the context to track the entity using the Attach method on DbSet. The entity will be in the Unchanged state in the context. For a better understanding, please have a look at the following example. The following example code is self-explained, so please go through the comment line.

```
using System;
using System.Data.Entity;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Creating an Existing Department Entity
                Department ExistingDepartment = new Department()
                {
                    ID = 1,
                    Name = "IT",
                    Location = "BBSR"
                };
                //Attaching the Entity to the Context
                context.Departments.Attach(ExistingDepartment);
                //Or you can also attach an existing entity as follows
                //context.Entry(ExistingDepartment).State =
                EntityState.Unchanged;
                //Check the Entity State Before Modifying
                Console.WriteLine($"Before SaveChanges Entity State:
                {context.Entry(ExistingDepartment).State}");
                context.SaveChanges();
                //Check the Entity State After Calling the SaveChanges Method
                of Context Object
                Console.WriteLine($"After SaveChanges Entity State:
                {context.Entry(ExistingDepartment).State}");
                Console.ReadKey();
            }
        }
    }
}
```

Output:

```
Before SaveChanges Entity State: Unchanged
After SaveChanges Entity State: Unchanged
```

Detached State of an Entity in Entity Framework with Example

We can change the Entity State to the Detached state by using the Entity State property. Once the entity is in the Detached state, it cannot be tracked by the Context object. We have to use Attach() method for the entity to be tracked by the Context. The Detached entity state indicates that the entity is not being tracked by the context. For a better understanding, please have a look at the following example. The following example code is self-explained, so please go through the comment line.

```
using System;
using System.Data.Entity;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Fetch the First Department
                Department firstDepartment = context.Departments.Find(1);
                //Check the Entity State Before Modifying
                Console.WriteLine($"Before Detached Entity State:
{context.Entry(firstDepartment).State}");

                //Changing the Entity State to Detached
                context.Entry(firstDepartment).State = EntityState.Detached;
                Console.WriteLine($"Entity State After Detached:
{context.Entry(firstDepartment).State}");

                firstDepartment.Location = "Location Changed";
                //The above changes will have no impact on the database
                context.SaveChanges();

                //Check the Entity State After Calling the SaveChanges Method
                of Context Object
                Console.WriteLine($"After SaveChanges Entity State:
{context.Entry(firstDepartment).State}");
                Console.ReadKey();
            }
        }
    }
}
```

Output:

```
Before Detached Entity State: Unchanged
Entity State After Detached: Detached
After SaveChanges Entity State: Detached
```

Added State of an Entity in Entity Framework with Example

Whenever we add a new Entity to the context object using the Add method, then the state of the entity will be in the Added state. Added entity state indicates that the entity exists in the context, but does not exist in the database. In this case, DbContext generates the INSERT SQL query and inserts the data into the database when the SaveChanges method is invoked. Once the SaveChanges method execution is successful, the state of the Entity is changed to an Unchanged state. For a better understanding, please have a look at the following example. The following example code is self-explained, so please go through the comment line.

```
using System;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Create a New Entity
                Department newDepartment = new Department()
                {
                    Name = "Sales",
                    Location = "Mumbai"
                };
                //Add the new Entity to the context object
                //Now context object tracking the entity state
                context.Departments.Add(newDepartment);
                //Check the Entity State Before Calling the SaveChanges
                Method of Context Object
                Console.WriteLine($"Before SaveChanges Entity State:
                {context.Entry(newDepartment).State}");
                //Add the Entity State in the Database by calling the Save
                Changes Method
                context.SaveChanges();
                //Check the Entity State After Calling the SaveChanges Method
                of Context Object
                Console.WriteLine($"After SaveChanges Entity State:
                {context.Entry(newDepartment).State}");
                Console.ReadKey();
            }
        }
    }
}
```

Output:

```
Before SaveChanges Entity State: Added
After SaveChanges Entity State: Unchanged
```

In this example, for the Added state, the Entity Framework will generate and execute the following SQL Statements. You can capture this using the SQL Profiler tool which is available in SSMS.

```
exec sp_executesql N'INSERT [dbo].[Departments]([Name], [Location])
VALUES (@0, @1)
SELECT [ID]
FROM [dbo].[Departments]
WHERE @@ROWCOUNT > 0 AND [ID] = scope_identity()',N'@0 varchar(50),@1
varchar(50)',@0='Sales',@1='Mumbai'
```

Modified State of an Entity in Entity Framework with Example

The entity will be in a Modified state whenever we modify scalar properties. The Modified entity state indicates that the entity is modified but not updated in the database. It also indicates that the entity exists in the database. The DbContext generates the UPDATE SQL Query to update the entity in the database. Once the saveChanges is successful the state of the entity is changed to Unchanged. For a better understanding, please have a look at the following example.

```
using System;
using System.Data.Entity;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Fetch the First Department
                Department firstDepartment = context.Departments.Find(1);
                Console.WriteLine($"Before Modifying Entity State:
{context.Entry(firstDepartment).State}");
                //Modify the Entity
                firstDepartment.Location = "BBSR UPDATED";
                Console.WriteLine($"After Modifying Entity State:
{context.Entry(firstDepartment).State}");
                //Update the Entity in the Database by calling the
                SaveChanges Method
                context.SaveChanges();
                Console.WriteLine($"After SaveChanges Entity State:
{context.Entry(firstDepartment).State}");
                Console.ReadKey();
            }
        }
    }
}
```

Output:

```
Before Modifying Entity State: Unchanged
After Modifying Entity State: Modified
After SaveChanges Entity State: Unchanged
```

In the above example, the Entity Framework will generate and execute the following SQL Statement.

```
exec sp_executesql N'UPDATE [dbo].[Departments]
SET [Location] = @0
WHERE ([ID] = @1)
',N'@0 varchar(50),@1 int',@0='BBSR UPDATED',@1=1
```

The Entity framework keeps track of the properties that have been modified. The Columns in the Update statement are set for only those columns, whose values are modified.

Deleted State of an Entity in Entity Framework with Example

Whenever we call the Remove method, the entity will be removed from the context and will be marked as a “Deleted” state. When the SaveChanges method is called, the corresponding rows are deleted from the database. The Deleted entity state indicates that the entity is marked for deletion, but not yet deleted from the database. It also indicates that the entity exists in the database. The DbContext generates the DELETE SQL Query to remove the entity from the database. The entity is removed from the context once the delete operation succeeds after the saveChanges. For a better understanding, please have a look at the following example.

```
using System;
using System.Data.Entity;
namespace EFDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            using (EF_Demo_DBEntities context = new EF_Demo_DBEntities())
            {
                //Fetch the First Department
                Department firstDepartment = context.Departments.Find(1);
                Console.WriteLine($"Before DELETING Entity State:
{context.Entry(firstDepartment).State}");
                //Modify the Entity
                context.Departments.Remove(firstDepartment);
                Console.WriteLine($"After DELETING Entity State:
{context.Entry(firstDepartment).State}");
                //Delete the Entity in the Database by calling the
                SaveChanges Method
                context.SaveChanges();
                Console.WriteLine($"After SaveChanges Entity State:
{context.Entry(firstDepartment).State}");
                Console.ReadKey();
            }
        }
    }
}
```

Output:

```
Before DELETING Entity State: Unchanged  
After DELETING Entity State: Deleted  
After SaveChanges Entity State: Detached
```

In the above example, the Entity Framework will generate and execute the following SQL Statement.

```
exec sp_executesql N'DELETE [dbo].[Departments]  
WHERE ([ID] = @0)',N'@0 int',@0=1
```

What SaveChanges Method Does in Entity Framework?

The SaveChanges method of context object does different things for entities in different states. They are as follows:

1. If the entities are in Unchanged State, then the SaveChanges method will not touch them. And this makes sense because the entities are not modified or added or deleted, then why should send to the database?
2. If the entities are in Added State, then those entities are inserted into the database when the SaveChanges method is called and once the SaveChanges method is called, then those entities are moved to the Unchanged state.
3. If the entities are in Modified State, then those entities are updated in the database when the SaveChanges method is called and once the SaveChanges method is called, then those entities are moved to Unchanged state.
4. If the entities are in Deleted State, then those entities are deleted from the database when the SaveChanges method is called and then, they are detached from the context object i.e. their state is not maintained by the context object anymore.

Note: The point that you need to remember is Entity Framework builds and executes the INSERT, UPDATE, and DELETE commands based on the state of an entity when the context.SaveChanges() method is called. It executes the INSERT command for the entities with Added state, executes the UPDATE command for the entities with the Modified state, and executes the DELETE command for the entities in the Deleted state. The context does not track entities in the Detached state because entities are not tracked by the context object.

In the next article, I am going to discuss [Development Approach with Entity Framework](#). Here, In this article, I try to explain the **Entity States in Entity Framework**. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article